

LABORATORY MANUAL

For

DATABASE MANAGEMENT SYSTEMS

Course Code: U25DSPC307

S.Y. BTECH

SEMESTER-III

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL NO. 7

WINDOW FUNCTIONS IMPLEMENTATION

1. Aim

To perform analytical queries using window functions in MySQL with OVER, PARTITION BY, RANK, DENSE_RANK, ROW_NUMBER, LEAD and LAG.

2. Problem Statement

The college wants analytical reports on student performance from *college_db* together with a new table *semester_result(student_id, semester, sgpa)*:

1. Show each student's marks together with the overall average, the course average, the gap from the course topper and the class size.
2. Rank students within each course, showing how ties are handled.
3. List the top two students of every course.
4. Show the SGPA of each semester with the previous semester's SGPA, the change and the next semester's SGPA.
5. Show the cumulative CGPA after each semester and a running total of instructor salaries.
6. Divide semester-4 results into quartiles and show percentile values.
7. Display the topper of every course.

Note: The course document lists only the title of this practical. The case study, sample data and individual tasks below were prepared for this report to fulfil that title; they are illustrative and are not official university questions. All names, e-mail addresses and phone numbers are fictitious.

3. Objectives

1. To understand the difference between window functions and GROUP BY aggregation.
2. To use OVER with PARTITION BY, ORDER BY and frame clauses.
3. To rank rows using ROW_NUMBER, RANK, DENSE_RANK and NTILE.
4. To access previous and next rows using LAG and LEAD.
5. To solve top-N-per-group and running-total problems.

4. Software / Tools Required

Sr. No.	Software / Tool	Purpose
1	MySQL Community Server 8.0 (version 8.0.46 used)	Relational DBMS on which all SQL statements were executed
2	MySQL command-line client (mysql)	Running the SQL scripts and recording the output
3	Operating system: Ubuntu 24.04 LTS (Windows 10/11 also suitable)	Platform for the DBMS server and client
4	Text editor (VS Code / Notepad++)	Writing and saving the .sql script files

5. Theory

5.1 Window Functions

A **window function** performs a calculation across a set of rows that are related to the current row - its *window* - and returns a value for **every** row. GROUP BY aggregation returns one row per group, whereas a window function keeps the individual rows, so detail values and summary values can be shown side by side. Window functions are supported in MySQL from version 8.0 (and in PostgreSQL, SQL Server and Oracle).

5.2 The OVER Clause

```
function_name([args]) OVER (
    [PARTITION BY expr, ...]      -- divide rows into independent groups
    [ORDER BY expr [ASC|DESC], ...] -- order of rows inside each partition
    [frame_clause]                -- ROWS | RANGE BETWEEN start AND end
)
-- named window: ... OVER w ... WINDOW w AS (PARTITION BY ... ORDER BY ...)
```

- **OVER ()** - the window is the whole result set.
- **PARTITION BY** - restarts the calculation for each partition (similar to GROUP BY but without collapsing rows).
- **ORDER BY** - required by ranking and offset functions; with aggregates it produces running totals because the default frame becomes RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.
- **Frame clause** - e.g. ROWS BETWEEN 2 PRECEDING AND CURRENT ROW for a moving window.

5.3 Categories of Window Functions

Category	Functions	Purpose
Aggregate	SUM, AVG, COUNT, MIN, MAX ... OVER (...)	Group totals, running totals, moving averages
Ranking	ROW_NUMBER, RANK, DENSE_RANK, NTILE(n)	Position of a row within its partition
Distribution	PERCENT_RANK, CUME_DIST	Relative standing between 0 and 1
Offset (value)	LAG(expr, n, default), LEAD(expr, n, default)	Value from the n-th previous / next row
Value	FIRST_VALUE, LAST_VALUE, NTH_VALUE	Value from a specific row of the frame

5.4 RANK versus DENSE_RANK versus ROW_NUMBER

Marks (desc)	ROW_NUMBER	RANK	DENSE_RANK
88	1	1	1
78	2	2	2
78	3	2	2
65	4	4	3

Window functions are evaluated after WHERE, GROUP BY and HAVING, just before ORDER BY. Therefore they cannot appear in WHERE; to filter on their result, the query is wrapped in a derived table or a CTE.

6. Procedure / Algorithm

1. Create and populate *college_db* (Q1, Q2) and create the *semester_result* table with SGPA data (Q3).
2. Use aggregate functions with OVER () and OVER (PARTITION BY course_id) to compare each student's marks with the overall and course averages.
3. Apply ROW_NUMBER, RANK and DENSE_RANK within each course and compare their handling of ties.
4. Find the top two students of each course using DENSE_RANK inside a CTE.
5. Use LAG and LEAD to compare each semester's SGPA with the previous and next semester.
6. Compute running values (CGPA so far, cumulative salary) using ORDER BY and frame clauses.
7. Use NTILE, PERCENT_RANK, CUME_DIST and FIRST_VALUE.
8. Observe that a window function is not allowed in WHERE and filter using a derived table.
9. Verify the output against the sample data.

7. SQL Queries / Program

Q1: Setup: create college_db and its tables (from Practical 2)

```
DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE department (
    dept_id      INT          NOT NULL,
    dept_code    VARCHAR(10)  NOT NULL,
    dept_name    VARCHAR(40)  NOT NULL,
    building     VARCHAR(20),
    budget       DECIMAL(12,2) DEFAULT 1000000.00,
    CONSTRAINT pk_department PRIMARY KEY (dept_id),
    CONSTRAINT uq_dept_code  UNIQUE (dept_code),
    CONSTRAINT chk_budget    CHECK (budget > 0)
);

CREATE TABLE instructor (
    instr_id      INT          NOT NULL,
    instr_name    VARCHAR(40)  NOT NULL,
    designation   VARCHAR(20)  NOT NULL,
    dept_id       INT,
    salary        DECIMAL(10,2) NOT NULL,
    email         VARCHAR(40)  NOT NULL,
    mentor_id     INT,
    CONSTRAINT pk_instructor PRIMARY KEY (instr_id),
    CONSTRAINT uq_instr_email UNIQUE (email),
    CONSTRAINT chk_salary    CHECK (salary BETWEEN 30000 AND 300000),
    CONSTRAINT fk_instr_dept FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_instr_mentor FOREIGN KEY (mentor_id)
        REFERENCES instructor (instr_id) ON DELETE SET NULL
);
```

```

);

CREATE TABLE student (
    student_id    INT            NOT NULL AUTO_INCREMENT,
    roll_no       VARCHAR(10)    NOT NULL,
    student_name  VARCHAR(40)    NOT NULL,
    gender        CHAR(1)        NOT NULL,
    dob           DATE           NOT NULL,
    dept_id       INT            NOT NULL,
    city          VARCHAR(30)     DEFAULT 'Pune',
    email         VARCHAR(40)     NOT NULL,
    CONSTRAINT pk_student        PRIMARY KEY (student_id),
    CONSTRAINT uq_roll_no        UNIQUE (roll_no),
    CONSTRAINT uq_student_email  UNIQUE (email),
    CONSTRAINT chk_gender        CHECK (gender IN ('M', 'F', 'O')),
    CONSTRAINT chk_dob           CHECK (dob >= '1995-01-01'),
    CONSTRAINT fk_student_dept   FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON UPDATE CASCADE
);

CREATE TABLE student_phone (
    student_id    INT            NOT NULL,
    phone         CHAR(10)       NOT NULL,
    CONSTRAINT pk_student_phone PRIMARY KEY (student_id, phone),
    CONSTRAINT chk_phone        CHECK (phone REGEXP '^[6-9][0-9]{9}$'),
    CONSTRAINT fk_phone_student FOREIGN KEY (student_id)
        REFERENCES student (student_id) ON DELETE CASCADE
);

CREATE TABLE course (
    course_id     CHAR(5)        NOT NULL,
    title         VARCHAR(40)    NOT NULL,
    dept_id       INT            NOT NULL,
    credits       TINYINT        NOT NULL,
    instr_id      INT,
    prereq_id     CHAR(5),
    CONSTRAINT pk_course        PRIMARY KEY (course_id),
    CONSTRAINT uq_course_title  UNIQUE (title),
    CONSTRAINT chk_credits      CHECK (credits BETWEEN 1 AND 5),
    CONSTRAINT fk_course_dept   FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON UPDATE CASCADE,
    CONSTRAINT fk_course_instr  FOREIGN KEY (instr_id)
        REFERENCES instructor (instr_id) ON DELETE SET NULL,
    CONSTRAINT fk_course_prereq FOREIGN KEY (prereq_id)
        REFERENCES course (course_id)
);

CREATE TABLE enrollment (
    student_id    INT            NOT NULL,
    course_id     CHAR(5)        NOT NULL,
    semester      TINYINT        NOT NULL,
    marks         INT,
    grade         CHAR(2),
    CONSTRAINT pk_enrollment    PRIMARY KEY (student_id, course_id),
    CONSTRAINT chk_semester     CHECK (semester BETWEEN 1 AND 8),
    CONSTRAINT chk_marks       CHECK (marks BETWEEN 0 AND 100),
    CONSTRAINT fk_enr_student   FOREIGN KEY (student_id)
        REFERENCES student (student_id) ON DELETE CASCADE,
    CONSTRAINT fk_enr_course    FOREIGN KEY (course_id) REFERENCES course (course_id)
);

```

This setup script recreates the common *college_db* case-study database (schema of Practical 2 and data of Practical 3) so that this practical can be executed independently on a fresh MySQL server.

Q2: Setup: insert the sample data (from Practical 3)

```

INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES
  (10, 'CSE', 'Computer Engineering', 'A Block', 2500000),
  (20, 'AIML', 'AI and Machine Learning', 'B Block', 3000000),
  (30, 'ECE', 'Electronics and Telecomm', 'C Block', 1800000),
  (40, 'MECH', 'Mechanical Engineering', 'D Block', 1500000),
  (50, 'CIVIL', 'Civil Engineering', 'D Block', 1200000);

INSERT INTO instructor VALUES
  (101, 'Anil Kulkarni', 'Professor', 10, 185000, 'anil.k@example.com', NULL),
  (102, 'Meera Joshi', 'Professor', 20, 172000, 'meera.j@example.com', 101),
  (103, 'Rahul Patil', 'Asst. Prof', 10, 82000, 'rahul.p@example.com', 101),
  (104, 'Sneha Deshmukh', 'Assoc. Prof', 20, 115000, 'sneha.d@example.com', 102),
  (105, 'Vikram Shinde', 'Professor', 30, 168000, 'vikram.s@example.com', 101),
  (106, 'Pooja Kale', 'Asst. Prof', 20, 78000, 'pooja.k@example.com', 104),
  (107, 'Nitin Pawar', 'Asst. Prof', 50, 76000, 'nitin.p@example.com', 105),
  (108, 'Kavita More', 'Assoc. Prof', 40, 112000, 'kavita.m@example.com', 105);

INSERT INTO student
  (roll_no, student_name, gender, dob, dept_id, city, email) VALUES
  ('S24CS001', 'Aarav Sharma', 'M', '2005-03-14', 10, 'Pune', 'aarav@example.com'),
  ('S24CS002', 'Isha Gupta', 'F', '2005-07-22', 10, 'Mumbai', 'isha@example.com'),
  ('S24AI001', 'Rohan Mehta', 'M', '2005-01-09', 20, 'Pune', 'rohan@example.com'),
  ('S24AI002', 'Priya Nair', 'F', '2004-11-30', 20, 'Nashik', 'priya@example.com'),
  ('S24AI003', 'Kunal Verma', 'M', '2005-05-18', 20, 'Pune', 'kunal@example.com'),
  ('S24EC001', 'Sneha Iyer', 'F', '2005-09-02', 30, 'Nagpur', 'sneha@example.com'),
  ('S24EC002', 'Aditya Rao', 'M', '2004-12-25', 30, 'Mumbai', 'aditya@example.com'),
  ('S24ME001', 'Neha Kulkarni', 'F', '2005-02-11', 40, 'Pune', 'neha@example.com'),
  ('S24CS003', 'Yash Jadhav', 'M', '2005-08-05', 10, NULL, 'yash@example.com'),
  ('S24AI004', 'Ananya Desai', 'F', '2005-04-27', 20, 'Mumbai', 'ananya@example.com');

INSERT INTO student_phone VALUES
  (1, '9876500011'), (1, '9876500012'), (2, '9876500021'), (3, '9876500031'),
  (4, '9876500041'), (4, '9876500042'), (6, '9876500061'), (9, '9876500091');

INSERT INTO course VALUES
  ('CS101', 'Programming in C', 10, 4, 103, NULL),
  ('CS201', 'Data Structures', 10, 4, 103, 'CS101'),
  ('CS301', 'Database Management Systems', 10, 3, 101, 'CS201'),
  ('CS401', 'Advanced Databases', 10, 3, 101, 'CS301'),
  ('AI201', 'Python for Data Science', 20, 3, 104, 'CS101'),
  ('AI301', 'Machine Learning', 20, 4, 102, 'AI201'),
  ('AI401', 'Deep Learning', 20, 3, 106, 'AI301'),
  ('EC201', 'Digital Electronics', 30, 3, 105, NULL),
  ('EC301', 'Signals and Systems', 30, 4, NULL, 'EC201'),
  ('ME201', 'Engineering Mechanics', 40, 3, 108, NULL);

INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES
  (1, 'CS101', 3, 78, 'A'), (1, 'CS201', 3, 85, 'A+'), (1, 'CS301', 4, 91, 'O'),
  (1, 'AI201', 3, 72, 'A'), (2, 'CS101', 3, 88, 'A+'), (2, 'CS201', 3, 67, 'B+'),
  (2, 'CS301', 4, 74, 'A'), (3, 'CS101', 3, 65, 'B+'), (3, 'AI201', 3, 81, 'A+'),
  (3, 'AI301', 4, 89, 'A+'), (3, 'CS301', 4, 70, 'A'), (4, 'AI201', 3, 92, 'O'),
  (4, 'AI301', 4, 76, 'A'), (4, 'AI401', 4, 84, 'A+'), (5, 'AI201', 3, 58, 'B'),
  (5, 'AI301', 4, 63, 'B+'), (6, 'EC201', 3, 79, 'A'), (6, 'CS101', 3, 55, 'B'),
  (6, 'EC301', 4, 82, 'A+'), (7, 'EC201', 3, 45, 'C'), (7, 'EC301', 4, 61, 'B+'),
  (8, 'CS101', 3, 78, 'A'), (9, 'CS201', 3, 90, 'O'), (9, 'CS301', 4, 95, 'O'),
  (9, 'CS401', 4, 88, 'A+');

```

Q3: Setup: semester-wise SGPA table used for LAG / LEAD

```

CREATE TABLE semester_result (
  student_id INT NOT NULL,
  semester TINYINT NOT NULL,
  sgpa DECIMAL(4,2) NOT NULL,
  PRIMARY KEY (student_id, semester),
  FOREIGN KEY (student_id) REFERENCES student (student_id),
  CHECK (sgpa BETWEEN 0 AND 10)
);
INSERT INTO semester_result VALUES
  (1, 1, 8.20), (1, 2, 8.60), (1, 3, 8.90),
  (2, 1, 8.80), (2, 2, 8.10), (2, 3, 8.40),
  (3, 1, 7.50), (3, 2, 7.90), (3, 3, 8.30),
  (4, 1, 9.10), (4, 2, 9.00), (4, 3, 9.40),
  (5, 1, 6.80), (5, 2, 6.50), (5, 3, 7.20);

```

Q4: Aggregate window functions with OVER() and OVER (PARTITION BY)

```

SELECT student_id, course_id, marks,
  ROUND(AVG(marks) OVER (), 2) AS overall_avg,
  ROUND(AVG(marks) OVER (PARTITION BY course_id), 2) AS course_avg,
  marks - MAX(marks) OVER (PARTITION BY course_id) AS behind_topper,
  COUNT(*) OVER (PARTITION BY course_id) AS class_size
FROM enrollment
WHERE course_id IN ('CS101', 'CS301')
ORDER BY course_id, marks DESC;

```

Unlike GROUP BY, a window function does not collapse rows: every enrollment row is kept and the aggregate of its window is added as an extra column. Window functions are evaluated after WHERE, so OVER () here covers the 9 selected rows of CS101 and CS301.

Q5: Ranking: ROW_NUMBER, RANK and DENSE_RANK within each course

```

SELECT course_id, student_id, marks,
  ROW_NUMBER() OVER w AS row_num,
  RANK() OVER w AS rnk,
  DENSE_RANK() OVER w AS dense_rnk
FROM enrollment
WHERE course_id IN ('CS101', 'AI201')
WINDOW w AS (PARTITION BY course_id ORDER BY marks DESC)
ORDER BY course_id, marks DESC;

```

In CS101 two students have 78 marks: RANK gives both rank 2 and skips rank 3, DENSE_RANK gives both rank 2 without a gap, and ROW_NUMBER numbers them 2 and 3 arbitrarily. The WINDOW clause defines the named window w once and reuses it.

Q6: Top-N per group: the two toppers of every course

```

WITH ranked AS (
  SELECT e.course_id, s.student_name, e.marks,
    DENSE_RANK() OVER (PARTITION BY e.course_id
      ORDER BY e.marks DESC) AS position
  FROM enrollment e JOIN student s ON s.student_id = e.student_id
)
SELECT course_id, position, student_name, marks
FROM ranked
WHERE position <= 2
ORDER BY course_id, position;

```

Q7: LAG and LEAD: change of SGPA from the previous semester and the next SGPA

```
SELECT student_id, semester, sgpa,
       LAG(sgpa) OVER (PARTITION BY student_id ORDER BY semester) AS prev_sgpa,
       sgpa - LAG(sgpa) OVER (PARTITION BY student_id ORDER BY semester) AS sgpa_change,
       LEAD(sgpa, 1, 0) OVER (PARTITION BY student_id ORDER BY semester) AS next_sgpa
FROM   semester_result
WHERE  student_id IN (1, 2, 5)
ORDER BY student_id, semester;
```

LAG returns NULL for the first semester because there is no previous row in the partition. LEAD(sgpa, 1, 0) supplies the default value 0 when there is no next row.

Q8: Running (cumulative) values with ORDER BY and frame clauses

```
SELECT student_id, semester, sgpa,
       ROUND(AVG(sgpa) OVER (PARTITION BY student_id ORDER BY semester
                             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 2) AS cgpa_so_far,
       MAX(sgpa) OVER (PARTITION BY student_id) AS best_sgpa
FROM   semester_result
WHERE  student_id IN (3, 4)
ORDER BY student_id, semester;

SELECT instr_id, instr_name, salary,
       SUM(salary) OVER (ORDER BY instr_id) AS running_total
FROM   instructor;
```

Q9: Distribution functions: NTILE, PERCENT_RANK, CUME_DIST and FIRST_VALUE

```
SELECT student_id, course_id, marks,
       NTILE(4) OVER (ORDER BY marks DESC) AS quartile,
       ROUND(PERCENT_RANK() OVER (ORDER BY marks), 2) AS pct_rank,
       ROUND(CUME_DIST() OVER (ORDER BY marks), 2) AS cum_dist,
       FIRST_VALUE(marks) OVER (ORDER BY marks DESC) AS top_marks
FROM   enrollment
WHERE  semester = 4
ORDER BY marks DESC;
```

Q10: Window functions cannot be used in WHERE (use a derived table or CTE)

```
SELECT student_id, course_id, marks
FROM   enrollment
WHERE  RANK() OVER (PARTITION BY course_id ORDER BY marks DESC) = 1;

SELECT course_id, student_id, marks
FROM   (SELECT course_id, student_id, marks,
              RANK() OVER (PARTITION BY course_id ORDER BY marks DESC) AS rnk
        FROM   enrollment) AS t
WHERE  rnk = 1
ORDER BY course_id;
```


8. Sample Input / Database Tables

The enrollment and student tables of *college_db* (created by Q1, Q2) and the semester_result table (Q3) are used. The values were read from the database after executing the setup queries.

Table: student (columns used)

student_id	roll_no	student_name	dept_id
1	S24CS001	Aarav Sharma	10
2	S24CS002	Isha Gupta	10
3	S24AI001	Rohan Mehta	20
4	S24AI002	Priya Nair	20
5	S24AI003	Kunal Verma	20
6	S24EC001	Sneha Iyer	30
7	S24EC002	Aditya Rao	30
8	S24ME001	Neha Kulkarni	40
9	S24CS003	Yash Jadhav	10
10	S24AI004	Ananya Desai	20

Table: enrollment (25 rows)

student_id	course_id	semester	marks	grade
1	AI201	3	72	A
1	CS101	3	78	A
1	CS201	3	85	A+
1	CS301	4	91	O
2	CS101	3	88	A+
2	CS201	3	67	B+
2	CS301	4	74	A
3	AI201	3	81	A+
3	AI301	4	89	A+
3	CS101	3	65	B+
3	CS301	4	70	A
4	AI201	3	92	O
4	AI301	4	76	A
4	AI401	4	84	A+
5	AI201	3	58	B
5	AI301	4	63	B+
6	CS101	3	55	B
6	EC201	3	79	A
6	EC301	4	82	A+
7	EC201	3	45	C
7	EC301	4	61	B+

student_id	course_id	semester	marks	grade
8	CS101	3	78	A
9	CS201	3	90	O
9	CS301	4	95	O
9	CS401	4	88	A+

Table: semester_result

student_id	semester	sgpa
1	1	8.20
1	2	8.60
1	3	8.90
2	1	8.80
2	2	8.10
2	3	8.40
3	1	7.50
3	2	7.90
3	3	8.30
4	1	9.10
4	2	9.00
4	3	9.40
5	1	6.80
5	2	6.50
5	3	7.20

Table: instructor (columns used)

instr_id	instr_name	salary
101	Anil Kulkarni	185000.00
102	Meera Joshi	172000.00
103	Rahul Patil	82000.00
104	Sneha Deshmukh	115000.00
105	Vikram Shinde	168000.00
106	Pooja Kale	78000.00
107	Nitin Pawar	76000.00
108	Kavita More	112000.00

9. Output

The output below is the actual text produced by the MySQL 8.0.46 command-line client when the statements of Section 7 were executed in batch mode (`mysql --table -vvv < script.sql`). Each result is preceded by the statement that produced it; statements longer than a few lines are abbreviated with '...'. Because the script ran in batch mode, error messages report the line number within the script file. Execution times shown in brackets depend on the machine.

Output of Q1: Setup: create college_db and its tables (from Practical 2)

```
mysql> DROP DATABASE IF EXISTS college_db;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.00 sec)

mysql> CREATE TABLE department ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE instructor ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student_phone ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE course ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE enrollment ( ...;
Query OK, 0 rows affected (0.02 sec)
```

Output of Q2: Setup: insert the sample data (from Practical 3)

```
mysql> INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES ...;
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> INSERT INTO instructor VALUES ...;
Query OK, 8 rows affected (0.00 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student_phone VALUES ...;
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO course VALUES ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES ...;
Query OK, 25 rows affected (0.00 sec)
Records: 25 Duplicates: 0 Warnings: 0
```

Output of Q3: Setup: semester-wise SGPA table used for LAG / LEAD

```
mysql> CREATE TABLE semester_result (
->   student_id INT NOT NULL,
->   semester TINYINT NOT NULL,
->   sgpa DECIMAL(4,2) NOT NULL,
->   PRIMARY KEY (student_id, semester),
->   FOREIGN KEY (student_id) REFERENCES student (student_id),
->   CHECK (sgpa BETWEEN 0 AND 10)
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO semester_result VALUES
->   (1, 1, 8.20), (1, 2, 8.60), (1, 3, 8.90),
->   (2, 1, 8.80), (2, 2, 8.10), (2, 3, 8.40),
->   (3, 1, 7.50), (3, 2, 7.90), (3, 3, 8.30),
->   (4, 1, 9.10), (4, 2, 9.00), (4, 3, 9.40),
->   (5, 1, 6.80), (5, 2, 6.50), (5, 3, 7.20);
Query OK, 15 rows affected (0.01 sec)
Records: 15 Duplicates: 0 Warnings: 0
```

Output of Q4: Aggregate window functions with OVER() and OVER (PARTITION BY)

```
mysql> SELECT student_id, course_id, marks,
->   ROUND(AVG(marks) OVER (), 2) AS overall_avg,
->   ROUND(AVG(marks) OVER (PARTITION BY course_id), 2) AS course_avg,
->   marks - MAX(marks) OVER (PARTITION BY course_id) AS behind_topper,
->   COUNT(*) OVER (PARTITION BY course_id) AS class_size
-> FROM enrollment
-> WHERE course_id IN ('CS101', 'CS301')
-> ORDER BY course_id, marks DESC;
```

student_id	course_id	marks	overall_avg	course_avg	behind_topper	class_size
2	CS101	88	77.11	72.80	0	5
1	CS101	78	77.11	72.80	-10	5
8	CS101	78	77.11	72.80	-10	5
3	CS101	65	77.11	72.80	-23	5
6	CS101	55	77.11	72.80	-33	5
9	CS301	95	77.11	82.50	0	4
1	CS301	91	77.11	82.50	-4	4
2	CS301	74	77.11	82.50	-21	4
3	CS301	70	77.11	82.50	-25	4

9 rows in set (0.00 sec)

Output of Q5: Ranking: ROW_NUMBER, RANK and DENSE_RANK within each course

```
mysql> SELECT course_id, student_id, marks,
->   ROW_NUMBER() OVER w AS row_num,
->   RANK() OVER w AS rnk,
->   DENSE_RANK() OVER w AS dense_rnk
-> FROM enrollment
-> WHERE course_id IN ('CS101', 'AI201')
-> WINDOW w AS (PARTITION BY course_id ORDER BY marks DESC)
-> ORDER BY course_id, marks DESC;
```

course_id	student_id	marks	row_num	rnk	dense_rnk
AI201	4	92	1	1	1
AI201	3	81	2	2	2
AI201	1	72	3	3	3
AI201	5	58	4	4	4
CS101	2	88	1	1	1
CS101	1	78	2	2	2
CS101	8	78	3	2	2
CS101	3	65	4	4	3
CS101	6	55	5	5	4

9 rows in set (0.00 sec)

Output of Q6: Top-N per group: the two toppers of every course

```
mysql> WITH ranked AS (
->     SELECT e.course_id, s.student_name, e.marks,
->           DENSE_RANK() OVER (PARTITION BY e.course_id
->                               ORDER BY e.marks DESC) AS position
->     ...
-> ORDER BY course_id, position;
+-----+-----+-----+-----+
| course_id | position | student_name | marks |
+-----+-----+-----+-----+
| AI201     | 1        | Priya Nair   | 92    |
| AI201     | 2        | Rohan Mehta  | 81    |
| AI301     | 1        | Rohan Mehta  | 89    |
| AI301     | 2        | Priya Nair   | 76    |
| AI401     | 1        | Priya Nair   | 84    |
| CS101     | 1        | Isha Gupta   | 88    |
| CS101     | 2        | Aarav Sharma | 78    |
| CS101     | 2        | Neha Kulkarni | 78    |
| CS201     | 1        | Yash Jadhav  | 90    |
| CS201     | 2        | Aarav Sharma | 85    |
| CS301     | 1        | Yash Jadhav  | 95    |
| CS301     | 2        | Aarav Sharma | 91    |
| CS401     | 1        | Yash Jadhav  | 88    |
| EC201     | 1        | Sneha Iyer   | 79    |
| EC201     | 2        | Aditya Rao   | 45    |
| EC301     | 1        | Sneha Iyer   | 82    |
| EC301     | 2        | Aditya Rao   | 61    |
+-----+-----+-----+-----+
17 rows in set (0.00 sec)
```

Output of Q7: LAG and LEAD: change of SGPA from the previous semester and the next SGPA

```
mysql> SELECT student_id, semester, sgpa,
->           LAG(sgpa) OVER (PARTITION BY student_id ORDER BY semester) AS prev_sgpa,
->           sgpa - LAG(sgpa) OVER (PARTITION BY student_id ORDER BY semester) AS
->           sgpa_change,
->           LEAD(sgpa, 1, 0) OVER (PARTITION BY student_id ORDER BY semester) AS next_sgpa
-> FROM semester_result
-> WHERE student_id IN (1, 2, 5)
-> ORDER BY student_id, semester;
+-----+-----+-----+-----+-----+-----+
| student_id | semester | sgpa | prev_sgpa | sgpa_change | next_sgpa |
+-----+-----+-----+-----+-----+-----+
| 1          | 1        | 8.20 | NULL      | NULL        | 8.60      |
| 1          | 2        | 8.60 | 8.20      | 0.40        | 8.90      |
| 1          | 3        | 8.90 | 8.60      | 0.30        | 0.00      |
| 2          | 1        | 8.80 | NULL      | NULL        | 8.10      |
| 2          | 2        | 8.10 | 8.80      | -0.70       | 8.40      |
| 2          | 3        | 8.40 | 8.10      | 0.30        | 0.00      |
| 5          | 1        | 6.80 | NULL      | NULL        | 6.50      |
| 5          | 2        | 6.50 | 6.80      | -0.30       | 7.20      |
| 5          | 3        | 7.20 | 6.50      | 0.70        | 0.00      |
+-----+-----+-----+-----+-----+-----+
9 rows in set (0.00 sec)
```

Output of Q8: Running (cumulative) values with ORDER BY and frame clauses

```
mysql> SELECT student_id, semester, sgpa,
->           ROUND(AVG(sgpa) OVER (PARTITION BY student_id ORDER BY semester
->                                ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 2) AS cgpa_so_far,
->           MAX(sgpa) OVER (PARTITION BY student_id) AS best_sgpa
-> FROM semester_result
-> WHERE student_id IN (3, 4)
-> ORDER BY student_id, semester;
+-----+-----+-----+-----+-----+
| student_id | semester | sgpa | cgpa_so_far | best_sgpa |
+-----+-----+-----+-----+-----+
| 3          | 1        | 7.50 | 7.50        | 8.30      |
```

```

|          3 |          2 | 7.90 |          7.70 |          8.30 |
|          3 |          3 | 8.30 |          7.90 |          8.30 |
|          4 |          1 | 9.10 |          9.10 |          9.40 |
|          4 |          2 | 9.00 |          9.05 |          9.40 |
|          4 |          3 | 9.40 |          9.17 |          9.40 |
+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

```

mysql> SELECT instr_id, instr_name, salary,
->      SUM(salary) OVER (ORDER BY instr_id) AS running_total
-> FROM    instructor;

```

```

+-----+-----+-----+-----+
| instr_id | instr_name | salary | running_total |
+-----+-----+-----+-----+
|      101 | Anil Kulkarni | 185000.00 | 185000.00 |
|      102 | Meera Joshi | 172000.00 | 357000.00 |
|      103 | Rahul Patil | 82000.00 | 439000.00 |
|      104 | Sneha Deshmukh | 115000.00 | 554000.00 |
|      105 | Vikram Shinde | 168000.00 | 722000.00 |
|      106 | Pooja Kale | 78000.00 | 800000.00 |
|      107 | Nitin Pawar | 76000.00 | 876000.00 |
|      108 | Kavita More | 112000.00 | 988000.00 |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)

```

Output of Q9: Distribution functions: NTILE, PERCENT_RANK, CUME_DIST and FIRST_VALUE

```

mysql> SELECT student_id, course_id, marks,
->      NTILE(4) OVER (ORDER BY marks DESC) AS quartile,
->      ROUND(PERCENT_RANK() OVER (ORDER BY marks), 2) AS pct_rank,
->      ROUND(CUME_DIST() OVER (ORDER BY marks), 2) AS cum_dist,
->      FIRST_VALUE(marks) OVER (ORDER BY marks DESC) AS top_marks
-> FROM    enrollment
-> WHERE   semester = 4
-> ORDER BY marks DESC;

```

```

+-----+-----+-----+-----+-----+-----+-----+
| student_id | course_id | marks | quartile | pct_rank | cum_dist | top_marks |
+-----+-----+-----+-----+-----+-----+-----+
|          9 | CS301 | 95 | 1 | 1 | 1 | 95 |
|          1 | CS301 | 91 | 1 | 0.9 | 0.91 | 95 |
|          3 | AI301 | 89 | 1 | 0.8 | 0.82 | 95 |
|          9 | CS401 | 88 | 2 | 0.7 | 0.73 | 95 |
|          4 | AI401 | 84 | 2 | 0.6 | 0.64 | 95 |
|          6 | EC301 | 82 | 2 | 0.5 | 0.55 | 95 |
|          4 | AI301 | 76 | 3 | 0.4 | 0.45 | 95 |
|          2 | CS301 | 74 | 3 | 0.3 | 0.36 | 95 |
|          3 | CS301 | 70 | 3 | 0.2 | 0.27 | 95 |
|          5 | AI301 | 63 | 4 | 0.1 | 0.18 | 95 |
|          7 | EC301 | 61 | 4 | 0 | 0.09 | 95 |
+-----+-----+-----+-----+-----+-----+-----+
11 rows in set (0.00 sec)

```

Output of Q10: Window functions cannot be used in WHERE (use a derived table or CTE)

```
mysql> SELECT student_id, course_id, marks
-> FROM enrollment
-> WHERE RANK() OVER (PARTITION BY course_id ORDER BY marks DESC) = 1;
ERROR 3593 (HY000) at line 222: You cannot use the window function 'rank' in this context.'
```

```
mysql> SELECT course_id, student_id, marks
-> FROM (SELECT course_id, student_id, marks,
->          RANK() OVER (PARTITION BY course_id ORDER BY marks DESC) AS rnk
->        FROM enrollment) AS t
-> WHERE rnk = 1
-> ORDER BY course_id;
```

course_id	student_id	marks
AI201	4	92
AI301	3	89
AI401	4	84
CS101	2	88
CS201	9	90
CS301	9	95
CS401	9	88
EC201	6	79
EC301	6	82

9 rows in set (0.00 sec)

10. Result

Window function queries were executed successfully in MySQL 8.0. OVER and PARTITION BY added overall and course-level aggregates without collapsing rows; ROW_NUMBER, RANK and DENSE_RANK produced the sequences 1-2-3-4-5, 1-2-2-4-5 and 1-2-2-3-4 for the tie in CS101; the top two students of each course were obtained; LAG and LEAD showed semester-to-semester SGPA changes; running CGPA and cumulative salary were computed with frame clauses; and MySQL rejected a window function in WHERE (ERROR 3593), which was solved using a derived table.

11. Conclusion

Window functions combine detail and summary information in a single query without self joins or correlated subqueries. PARTITION BY defines independent groups, ORDER BY defines the sequence for ranking, offsets and running totals, and the frame clause controls exactly which rows are aggregated. They are the standard solution for rankings, top-N per group, period-over-period comparison (LAG/LEAD) and cumulative values, and are available in MySQL from version 8.0.